

EE-170L Computer Programming Lab

Laboratory Report — No. 08

Two-Dimensional Arrays in C++

Student Name	Zawar Shah
Registration No.	BF25NWELE0677
Course	EE-170L – Computer Programming Lab
Lab No.	8
Topic	2D Arrays, Matrix Operations, Nested Loops
Semester	Spring 2026
Institution	UET Peshawar

1. Lab Objectives

- Understand the structure of a 2D array as a table of rows and columns.
- Declare and initialize 2D arrays using nested braces and flat initialization.
- Access individual elements using row and column subscripts: `a[i][j]`.
- Use nested for loops to traverse, display, and compute over 2D arrays.
- Calculate row sums, column sums, and overall totals from a matrix.
- Find the maximum value in a 2D array using a running-max variable.

2. Theoretical Background

2.1 What is a Two-Dimensional Array?

A two-dimensional array is the simplest form of a multidimensional array. It is essentially a list of one-dimensional arrays arranged in rows and columns — conceptually a table or matrix. It is used to store grid-structured data such as marks tables, game boards, or matrices.

Syntax:

```
type arrayName[rows][cols];

// Example: 3 rows, 4 columns
int a[3][4];
```

Each element is identified by two indices: `a[i][j]`, where `i` is the row (0-based) and `j` is the column (0-based).

2.2 Initializing Two-Dimensional Arrays

A 2D array can be initialized with nested braces (one set per row) or as a flat list — both produce identical results:

```
// Nested braces (preferred):
int a[3][4] = {
    {0, 1, 2, 3},
    {4, 5, 6, 7},
    {8, 9, 10, 11},
};
```

```

    {8, 9, 10, 11}
};

// Flat initialization (equivalent):
int a[3][4] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11};

```

2.3 Accessing Two-Dimensional Array Elements

Individual elements are accessed using two subscripts: `a[row][col]`. Nested for loops are the standard way to traverse all elements:

```

#include <iostream>
using namespace std;
int main()
{
    int a[5][2] = {{0,0},{1,2},{2,4},{3,6},{4,8}};
    for (int i = 0; i < 5; i++)
        for (int j = 0; j < 2; j++)
            cout << "a[" << i << "][" << j << "]: " << a[i][j] << endl;
    return 0;
}

```

Output:

```

a[0][0]: 0
a[0][1]: 0
a[1][0]: 1
a[1][1]: 2
a[2][0]: 2
a[2][1]: 4
a[3][0]: 3
a[3][1]: 6
a[4][0]: 4
a[4][1]: 8

```

2.4 Filling a Matrix Using Nested Loops

Nested for loops can also be used to fill a 2D array with user input or computed values:

```

#include <iostream>
using namespace std;
int main()
{
    int matrix[2][3];
    for (int m1 = 0; m1 < 2; m1++)
    {
        for (int m2 = 0; m2 < 3; m2++)
        {
            cout << "Enter Integer: ";
            cin >> matrix[m1][m2];
        }
    }
    for (int m1 = 0; m1 < 2; m1++)
        for (int m2 = 0; m2 < 3; m2++)
            cout << matrix[m1][m2] << "\t";
    return 0;
}

```

3. Lab Tasks & Solutions

Task 1 — 2D Array Declaration and Initialization

Declare a 3x3 integer array 'matrix' initialized with values 1 through 9. Display all elements using nested loops.

```

#include <iostream>

```

```
using namespace std;
int main()
{
    int matrix[3][3] = {
        {1, 2, 3},
        {4, 5, 6},
        {7, 8, 9}
    };
    cout << "Matrix:" << endl;
    for (int i = 0; i < 3; i++)
    {
        for (int j = 0; j < 3; j++)
            cout << matrix[i][j] << "\t";
        cout << endl;
    }
    return 0;
}
```

Output:

```
Matrix:
1    2    3
4    5    6
7    8    9
```

Task 2 — Sum of Rows and Columns in a 3x3 Array

Declare and initialize a 3x3 integer array. Calculate and display the sum of each row, each column, and the overall total.

```
#include <iostream>
using namespace std;
int main()
{
    int m[3][3] = {{3,7,2},{5,1,8},{4,6,9}};
    int total = 0;
    for (int i = 0; i < 3; i++)
    {
        int rowSum = 0;
        for (int j = 0; j < 3; j++) rowSum += m[i][j];
        cout << "Row " << i << " sum: " << rowSum << endl;
        total += rowSum;
    }
    for (int j = 0; j < 3; j++)
    {
        int colSum = 0;
        for (int i = 0; i < 3; i++) colSum += m[i][j];
        cout << "Col " << j << " sum: " << colSum << endl;
    }
    cout << "Total sum: " << total << endl;
    return 0;
}
```

How it works:

- First nested loop calculates and prints the sum of each row.
- Second nested loop calculates and prints the sum of each column.
- total accumulates all row sums, giving the overall array sum (= 45).

Task 3 — Finding the Maximum Value in a 4x4 Array

Declare and initialize a 4x4 integer array. Find and display the maximum value using nested loops and a running maximum variable.

```
#include <iostream>
```

```
using namespace std;
int main()
{
    int m[4][4] = {
        {12, 7, 43, 2},
        {99, 31, 8, 56},
        {5, 72, 14, 37},
        {21, 3, 88, 60}
    };
    int maxVal = m[0][0];
    for (int i = 0; i < 4; i++)
        for (int j = 0; j < 4; j++)
            if (m[i][j] > maxVal) maxVal = m[i][j];
    cout << "Maximum value: " << maxVal << endl;
    return 0;
}
```

Output:

```
Maximum value: 99
```

How it works:

- maxVal is initialized to m[0][0] as the starting reference.
- Nested loops compare every element to maxVal; whenever a larger value is found, maxVal is updated.
- After both loops finish, maxVal holds the largest element in the entire array.

4. 1D Array vs 2D Array — Key Differences

Feature	1D Array	2D Array
Structure	Single row of elements	Table with rows and columns
Declaration	int a[5];	int a[3][4];
Access	a[i]	a[i][j]
Traversal	Single for loop	Nested for loops
Use case	Lists, sequences	Matrices, grids, tables
Memory	n contiguous cells	n×m contiguous cells (row-major)

5. Conclusion

This lab introduced two-dimensional arrays, extending 1D arrays into a grid structure for matrix-style data. Key takeaways:

- **2D Arrays:** Represent tables with rows and columns; each element accessed via a[i][j].
- **Initialization:** Can use nested braces per row or a flat comma-separated list — both are equivalent.
- **Nested Loops:** Essential for traversal — outer loop for rows, inner loop for columns.

- **Matrix Operations:** Row/column sums, max/min search, and matrix arithmetic are all natural 2D array applications.
- **Memory Layout:** Elements are stored in contiguous row-major order, making access patterns important for performance.

2D arrays form the foundation for matrix arithmetic, image processing, and grid-based algorithms covered in advanced labs.